Sandia National Laboratories

Operated for the U.S. Department of Energy by
Sandia Corporation

Albuquerque, New Mexico 87185-

date: April 24, 2006

to: Distribution

from: David Noble, 1513, MS0826

subject: Tutorial on time step parameter selection for level set problems in GOMA (GT-034)

Introduction

Numerous parameters exist in GOMA that effect the time step size selection. Typically we seek to maximize the size of the time step while maintaining accuracy and stability. Each of these parameters is aimed at trying to accomplish this goal. In loosely coupled level set problems parameter selection becomes even more difficult due to the time scales introduced by the decoupling.

This tutorial examines two flows, a static bubble and a falling droplet, and explains the role of each time step parameter in these problems. The static bubble problem was studied extensively in a recent level set paper (Smolianski, 2005). The droplet falling into a shallow pool was examined by Chessa and Belytschko, 2003.

The discussion here will use many ideas from the paper by Smolianksi, 2005. It is useful to note some of the differences between the methods used in GOMA and the FELSOS method discussed in that paper. The most important difference is that GOMA solves the continuity and momentum equations using a fully coupled, full Newton approach. Only the level set advection is loosely coupled to that system. Specifically, GOMA lags the velocity in the level set advection equation. In contrast, FELSOS uses a time step made up of numerous loosely coupled steps. As a result, the time step selection in FELSOS must take into account the time scales introduced by each of these loosely coupled steps. In GOMA the attention is focused on the issues surrounding the time scales introduced by the loose coupling of the level set equation. Another difference between these methods involves the element types. The operator splitting used in FELSOS allows equal order, Q1Q1, elements, but suffers from issues that come from decoupling the viscous and pressure terms in the momentum equations. The simulations performed here using GOMA use a Q1P0 element with piecewise linear velocity and discontinuous pressure or a Q2Q1 element with piecewise quadratic velocity and piecewise linear pressure. The Q1P0 element is technically not LBB compliant but can offer reasonable performance in practice. Also, the GOMA simulations use XFEM to enrich the pressure to capture the discontinuous pressure due to the capillary force. It expected that the spurious velocities should be significantly lower than those obtained by Smolianski, 2005, for this reason.

The parameters that are discussed in this tutorial are:

Maximum time step
Courant Number Limit
Time step error
Minimum Resolved Time Step

The static bubble problem does not lend itself to a study of the parameter, **Minimum Resolved Time Step**. The falling droplet, however, does use this important parameter. Note that all of this discussion pertains to simulations that utilize loose coupling of the level set equations via the GOMA command:

Fill Weight Function = Explicit

Static Bubble Problem Description

A bubble at rest is simulated with multiple combinations of the above parameters. The physical properties are selected such that surface tension dominates viscosity. Following Smolianksi, 2005, a Laplace number is defined as,

$$La = \frac{2R\sigma\rho}{\mu^2}$$

where R is the radius of the drop, σ is the surface tension, ρ is the density, and μ is the viscosity. All of the simulations of the static bubble are performed at $La = 5.0 \times 10^3$. This is a useful parameter for this problem, which does not contain an intrinsic velocity scale. The magnitude of the spurious velocities are then measured by computing the capillary number defined as,

$$Ca = \frac{U\mu}{\sigma}$$

where U is computed as the maximum speed in the domain.

As discussed by Smolianski, 2005, there are both physical and numerical time scales present in this problem. To obtain a typical time scale for capillary driven flow they find the time interval which makes $Ca \approx 1$, with the velocity given by $U^{(ca)} = L \left| \Delta t_{phys}^{(ca)} \right.$, where L is the pertinent length scale in the problem. This gives a physically based limit for the time step,

$$\Delta t_{phys}^{(ca)} \approx \frac{\mu L}{\sigma}$$

In order to accurately model capillary driven flow, this physical time scale must be resolved, i.e. the time step must remain less than this physical time scale. This would remain a limiting factor for level set simulations even if a fully coupled method were used. Because of the loose coupling, however, an additional, numerical time step limit is present in this problem. According to the empirical analysis in Smolianksi, 2005 and its references, a numerical time step limit for capillary driven flow is given by,

$$\Delta t_{num}^{(ca)} = \sqrt{\frac{\rho}{\sigma}} h^{3/2}$$

where h is the mesh size. In many simulations, including the static bubble problem studied here (where $h = 1/40$), this is the controlling limit.

Static Bubble Simulations

From the above discussion the following definitions are given in the file **bubble.def:**

```
# La =              {La = 5.e3}
# L =               {L = 0.5}
# h =               {h = 1./40.}
# viscosity =       {viscosity =  1.}
# density =         {density = 1.}
# surface_tension = {surface_tension = La*viscosity*viscosity/density/L}
# dt_phys_ca =      {dt_phys_ca = L * viscosity / surface_tension}
# dt_num_ca =       {dt_num_ca = sqrt(density*h*h*h/surface_tension)}
```

The starting point for the tests uses the parameters:

```
Maximum time step                 = {dt_phys_ca}
Courant Number Limit              = 0.2
Time step error                   = -1.0 0 1 0 0 0 0 0
```

The **`Maximum time step`** states that we don't want to allow the time step to get larger than the physical time scale. In addition, we are relying on the **`Courant Number Limit`** to cut the time step if it would cause us to exceed the specified value of **`0.2`**. This card imposes an upper limit on the time step size, irrespective of the variable time integrator already in place. This upper limit is computed from the following relation:

$$dt_{\text{limit}} = C \min_{e} \left| \frac{h_e}{\left\| \hat{U} \right\|_e} \right|$$

where on the element, e , h_e is the average size of the element, C is the specified limit (**`0.2`**) and

$$\hat{U}_e = \frac{\int_e \delta_\alpha(\phi) v \cdot n d\Omega}{\int_e \delta_\alpha(\phi) d\Omega}$$

By specifying the **`Time step error`** of **-1.0**, we are basically turning off any time step control based on the deviation between the predictor and corrector. The GOMA manual provides further discussion of this parameter. Therefore we are solely relying on the **`Courant Number Limit`** to cut the time step for any numerical instability.

The Capillary number as a function of time is given in Figure 1. This first simulation is denoted as "Base Case" in the figure, and the value obtained by Smolianski, 2005, at steady state is shown by the horizontal line. Since this bubble should be stationary, the capillary number is a measure of the spurious velocities. While the magnitude of these spurious velocities is not much larger than that obtained by Smolianksi, 2005, it is much larger than expected. Smolianksi, 2005, did not use XFEM, which has been shown to significantly reduce the magnitude of the spurious velocities (GT-020). Also, the velocity field is highly erratic in time, without a clear steady state.

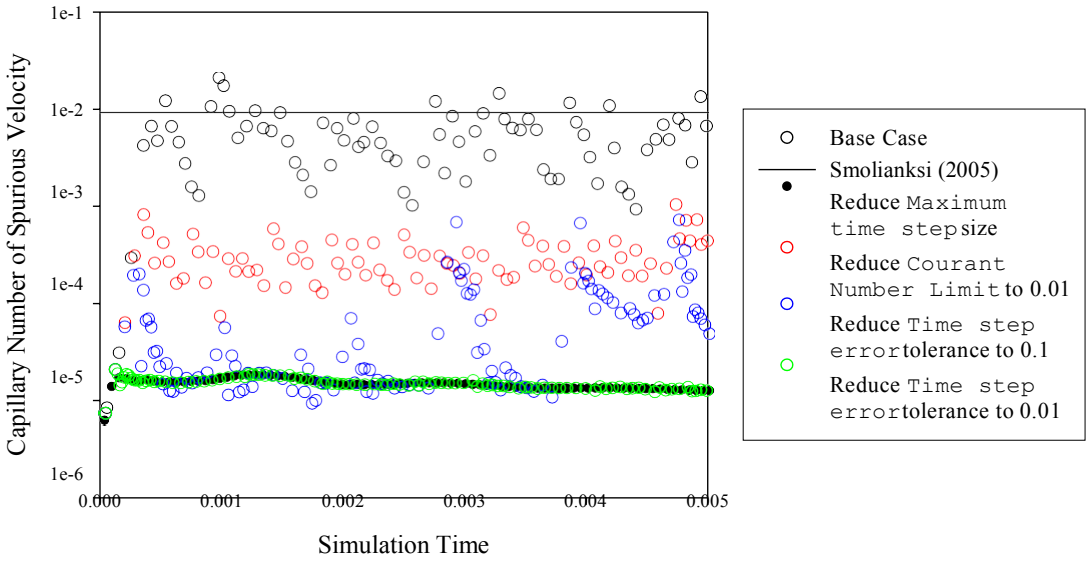

Figure 1. Spurious velocity for static bubble problem. Line shows steady state value reported by Smolianski, 2005. Symbols show results obtained by GOMA using different sets of time step parameters.

Controlling Accuracy by Reducing the Maximum Time Step

The first question is whether this error can be reduced by reducing the **`Maximum time step`**. The static bubble simulation was performed again, but with the parameters:

```
#dt_max = {dt_max=0.75*min(dt_num_ca,dt_phys_ca)}
Maximum time step                    = {dt_max}
Courant Number Limit                 = 0.2
Time step error                      = -1.0 0 1 0 0 0 0 0
```

The time step is now limited to a factor of safety times the minimum of the physical and numerical time scales. While the error is dramatically reduced as shown in Figure 1, the factor of safety, **0.75**, had to be determined empirically. In performing the simulations for this tutorial for various initial conditions and meshes, I found that sometimes this factor of safety would have to be lowered to 0.25 to provide low spurious velocities. This leads to the question of whether the other parameters might be able to provide accurate simulations without having to empirically determine a safe value of the **`Maximum time step`**.

Controlling Accuracy by Reducing the Courant Number Limit

Next we test if significantly reducing the **`Courant Number Limit`** can inhibit the spurious velocities. The static bubble simulation was performed again, now with the parameters:

```
#dt_max = {dt_max=1.0*min(dt_num_ca,dt_phys_ca)}
Maximum time step                    = {dt_max}
Courant Number Limit                 = 0.01
Time step error                      = -1.0 0 1 0 0 0 0 0
```

As discussed above, this will cut back the time step when it exceeds 1/100th of the mesh size divided by the velocity magnitude. While Figure 1 shows that the magnitude of the spurious velocities is reduced somewhat, they are not adequately controlled, even by this stringent specification. If this flow involved a steady motion of an interface this restriction would have been overly restrictive without providing any extra control over spurious velocity modes. This is clearly not the way to control the accuracy.

Controlling Accuracy by Reducing the Time Step Error

Next we test if the spurious velocities can be controlled using the **`Time step error`** tolerance. The static bubble simulation was performed twice more, now with the parameters:

```
#dt_max = {dt_max=1.0*min(dt_num_ca,dt_phys_ca)}
Maximum time step                  = {dt_max}
Courant Number Limit               = 0.2
Time step error                    = -0.10 0 1 0 0 0 0 0
```

followed by a run with the parameters:

```
#dt_max = {dt_max=1.0*min(dt_num_ca,dt_phys_ca)}
Maximum time step                  = {dt_max}
Courant Number Limit               = 0.2
Time step error                    = -0.01 0 1 0 0 0 0 0
```

Figure 1 clearly shows that a 10% time step error isn't quite tight enough to control the development of spurious velocities, while a 1% tolerance does inhibit their development.

To summarize this study, the most effective ways to reduce the errors caused by too large of time steps is to reduce the **`Maximum time step`** size or reduce the **`Time step error`** tolerance. Reducing the **`Courant Number Limit`** is an extremely ineffective way to control the error. The question remains whether it is better to reduce the **`Maximum time step`** size or the **`Time step error`** tolerance. The answer may be problem dependent, but the numerical experiments performed thus far indicate that **`Time step error`** provides better control while costing less. Reducing the **`Maximum time step`** size can significantly increase the cost of a simulation even when the system is reasonably stable. On the other hand, the **`Time step error`** can only react to the temporal instabilities that can be observed by comparing the predictor and corrector. If these modes are to be completely eliminated one must select a **`Maximum time step`** that will completely avoid generating these modes.

Anyone who has tried to rely on the **`Time step error`** for controlling the time step size knows what can go wrong, however. Before too long, the flow evolves to a geometry that involves some sudden change that causes the time step to enter the death spiral. The error tolerance is not met so the time step fails, and fails, and fails…

This leads to a discussion of the **`Minimum Resolved Time Step`** size. Its role is to set a lower bound for the time step with respect to the **`Time step error`** tolerance. When a converged time step is obtained by GOMA, the difference between the predicted solution and final solution for that time step is compared to the **`Time step error`** tolerance. If the

difference exceeds this tolerance the step fails and the time step is cut (usually by a factor of 2), UNLESS the time step falls below the **`Minimum Resolved Time Step`** size. In this case the step is accepted, even if this error tolerance is not achieved. This provides a mechanism for the modeler to control what phenomena is resolved and what phenomena is ignored. This parameter is not pertinent to the static bubble problem since the geometry does not change but does affect dynamic simulations like the falling droplet examined next.

Falling Droplet Problem Description

A droplet is simulated as it falls into a shallow pool. This problem was explored by Chessa and Belytschko, 2003. A time sequence of the simulation is shown in Figure 2. A drop of radius 0.25 accelerates under the force of gravity until it impacts a shallow pool of depth 0.25. The material properties are given in the section below.

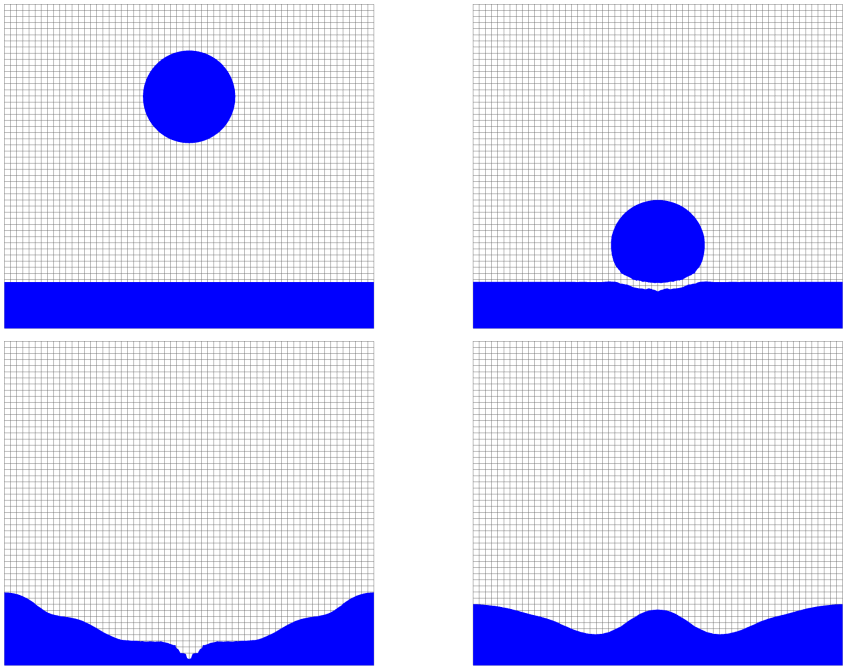

Figure 2. Time sequence for drop falling into a shallow pool. The simulations times are 0 for top left, 0.43 for top right, 0.75 for bottom left, and 0.92 for bottom right. The grid size for this simulation is 0.03333.

Falling Droplet Simulations

The following set of physical parameters were defined for the falling drop simulations:

```
# L =                {L = 0.5}
# h =                {h = 0.0125}
# viscosity1 =       {viscosity1 = 0.001}
# viscosity2 =       {viscosity2 = 0.002}
# density1 =         {density1 = 2.0}
# density2 =         {density2 = 0.1}
# surface_tension =  {surface_tension = 1.0}

# dt_phys_ca={dt_phys_ca= L*min(viscosity1,viscosity2)/surface_tension}
# dt_num_ca={dt_num_ca=sqrt(min(density1,density2)*h*h*h/surface_tension)}
```

The following set of time step parameters were then tested:

```
# courant_limit =   {courant_limit = 0.20}
# dt_max =          {dt_max = 5.*dt_phys_ca}
# dt_min =          {dt_min = 0.2*min(dt_num_ca,dt_phys_ca)}
# time_step_error = {time_step_error = 0.02}
delta_t                     = {dt_max}
Minimum time step           = {dt_min/10}
Maximum time step           = {dt_max}
Minimum Resolved Time Step  = {dt_min}
Courant Number Limit        = {courant_limit}
Time step error             = {-time_step_error} 0 1 0 0 0 0 0
```

These will be explained in reverse order. The **`Time step error`** was set to 0.02 based on the results obtained in the static drop problem. Adjusting this parameter will affect the cost and accuracy of the simulations with the cost increasing as the parameter is lowered, while time accuracy is increased. I think that it should be in the range between 0.01 and 0.1 for most problems. For problems in which very high accuracy is required, this parameter might be even smaller. Next, the **`Courant Number Limit`** is set to 0.2 to maintain that the level set advection equation is handled accurately with its explicit form. This droplet problem involves an accelerating droplet so it is important in this simulation that the Courant limit not be exceeded. I think that this parameter should be in the range of 0.1 to 0.5 for all problems. Next the **`Minimum Resolved Time Step`** is set to 0.2 times the minimum time scale in the problem. This states that it is ok for GOMA not to resolve phenomena that has a time scale less than this limit. Assuming that the physical and numerical time scales are known, I think this coefficient should be in the range of 0.1 to 0.5 for most problems. After all, why try to resolve some phenomena that are more than 10 times smaller than your physical and numerical time scales? Next the **`Maximum time step`** is set to 5 times the physical time scale. This upper bound is set so that, even if everything is completely smooth, we don't miss something that is occurring on the physical time scale. I think this coefficient should be somewhere in the range of 1 to 10 for most problems. Anything higher would run the risk of missing important phenomena, and anything lower would be unnecessarily expensive. Although we have specified the **`Minimum Resolved Time Step`**, we still need to specify the **`Minimum time step`** size. This is because the time step will continue to drop below the **`Minimum Resolved Time Step`** size if the Newton step fails to converge (as compared to the **`Time step error`** criterion failing). If for some reason the Newton steps will not converge and the time step is cut to less than the **`Minimum time step`** size, the simulation will halt with an error. Here I have specified that if GOMA can't obtain a converged solution even with a time step that is $1/10^{th}$ of the **`Minimum Resolved Time Step`** size, then the simulation fails. Hopefully this will never be reached.

Last we have the specification of the initial time step size, **`delta_t`**. Here I have optimistically set this to the **`Maximum time step`** size. This is appropriate in the falling droplet problem because it is accelerating from a zero velocity, so not much is happening when the simulation begins. For other problems, the initial phenomena are very fast and a shorter initial time step is needed. There is a second reason, however, that I have chosen a large initial time step. Currently, whenever renormalization is done, the time step is reset to its initial size if the current time step is larger than this size. This can unnecessarily shrink

the time step size if the initial time step is very small. I recommend that this behavior be revisited in the future. I am currently of the opinion that we can allow the normal **`Time step error`** evaluation catch any discontinuities caused by the renormalization and cut the time step if necessary. Under the current algorithm, I recommend that **`delta_t`** be set to as large of a value as the simulation will allow.

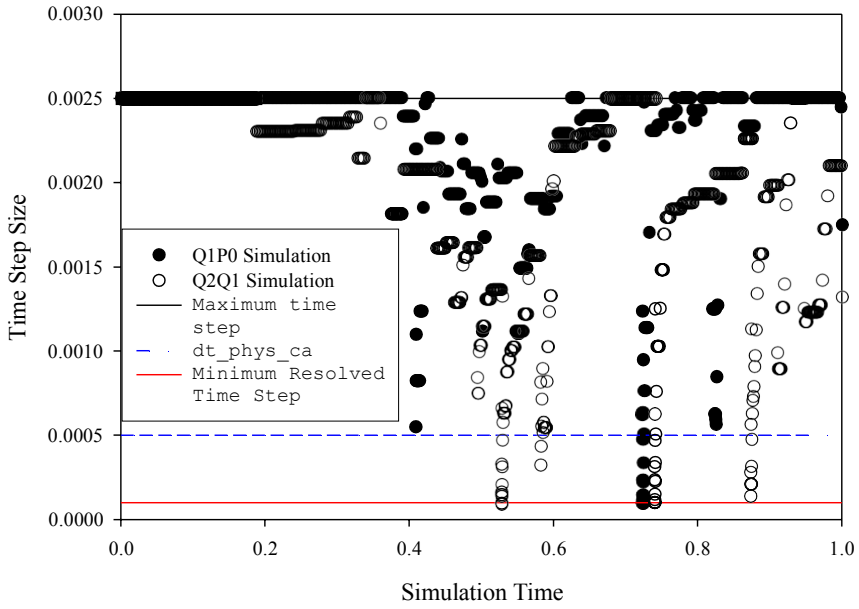

Figure 3. Time step size for the simulation of the falling drop with a grid size of 0.06666. The `dt_phys_num` is not shown because it exceeds the `Maximum time step` size. The excursions between `dt_phys_ca` and the `Minimum Resolved Time Step` size are examples where the `Time step error` tolerance is not met even for very small time step sizes. The `Minimum Resolved Time Step` flag is needed to keep these excursions from causing the death spiral.

Simulation Results

The above parameters were used in each of the 4 simulations. 3 simulations were performed using Q1P0 elements, with grid sizes of 0.0125, 0.03333, and 0.06666. Q2Q1 elements were used 1 simulation with a grid size of 0.06666. Figure 3 shows the time step size over the course of the simulations for the grid size of 0.06666 with both Q1P0 and Q2Q1 elements. Figure 4 shows the time step size for the Q1P0 simulation with a grid size of 0.03333. Finally, Figure 5 shows the time step size for the Q1P0 simulation with a grid size of 0.0125. Also included on each plot are 4 lines that correspond to the **`Maximum time step`**, **`dt_phys_ca`**, **`dt_num_ca`**, and **`Minimum Resolved Time Step`**. The **`dt_num_ca`** is left off of the plot for the grid size of 0.06666, because this time is larger than the **`Maximum time step`** size.

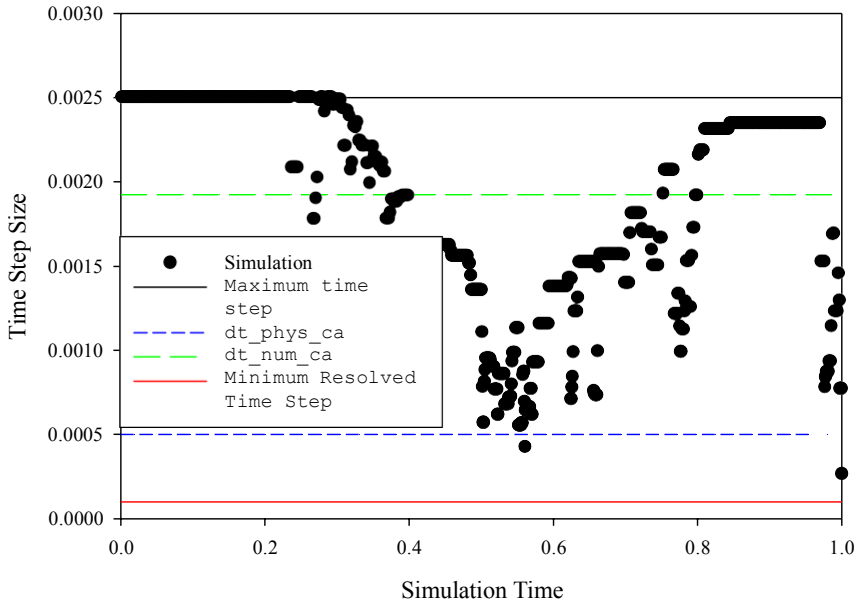

Figure 4. Time step size for the simulation of the falling drop with a grid size of 0.0333.

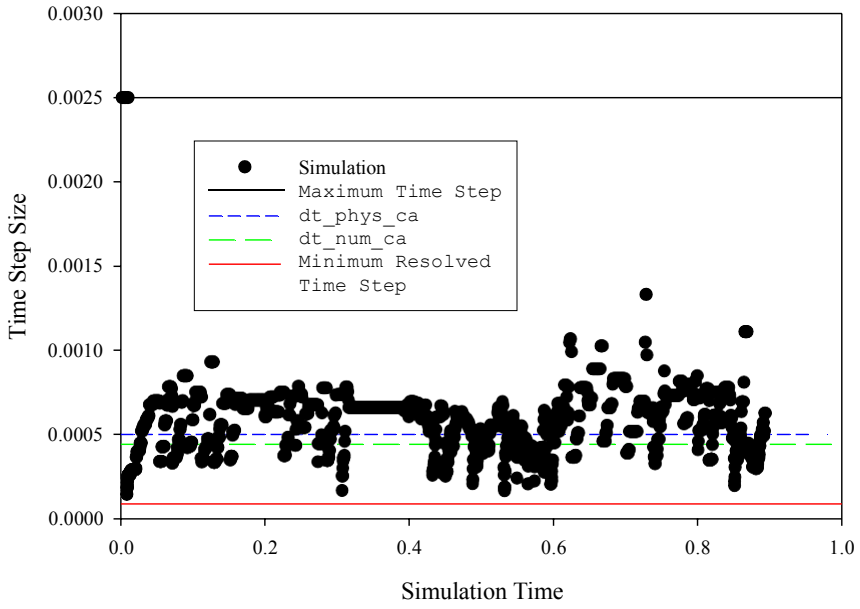

Figure 5. Time step size for the simulation of the falling drop with a grid size of 0.0125.

The results for the middle grid size of 0.03333, shown in Figure 4, will be discussed first. It is seen that the time step is equal to the **`Maximum time step`** only at the beginning of the simulation when the drop is accelerating, prior to collision. For most of the simulation, the time step falls in a band defined fairly well by the **`dt_phys_ca`** and **`dt_num_ca`**. This shows that the **`Time step error`** is being effective at sensing the intrinsic time scale of the phenomena and keeping the time step small enough to resolve it. For this grid size, the

Time step error never tries to cut the time step to less than **Minimum Resolved Time Step**, so this lower limit is not really used.

Next we examine the results for the coarse simulations, which have a grid size of 0.06666. For this case the **dt_num_ca** is larger. In fact it is larger than the **Maximum time step** size, so it is not shown in Figure 3. Again, the **Time step error** is effective at keeping the time step near enough to the **dt_phys_ca** so that the physical phenomena is well resolved temporally. For this grid size, however, there are a couple of times when the **Time step error** fails to achieve the desired accuracy even when the time step is cut below the **Minimum Resolved Time Step** size. The lower bound is imposed, however, and the simulation quickly recovers, with the time step again growing to a size on the order of the **dt_phys_ca**. Without the **Minimum Resolved Time Step** flag, these simulations would have entered the death spiral of ever decreasing time steps and failed. The flag is effective at keeping these simulations on track without trying to resolve unphysical oscillations due to inadequate resolution.

The fine simulations with a grid size of 0.0125 are shown in Figure 5. Here **dt_phys_ca** and **dt_num_ca** are nearly the same. Even in this very different regime, the **Time step error** is effective at keeping the time step in the vicinity of these time scales.

It is also noted that the **Courant Number Limit** is also occasionally the limiting factor for these simulations. While this flag is not effective at controlling the overall time integration error, it does help assure that the advection equation is modeled accurately.

Summary and Conclusions

The static bubble simulations showed that the spurious velocities can be controlled by reducing the **Maximum time step** size or lowering the **Time step error** tolerance. The latter is recommended because it is easier to select a value (between 0.01 and 0.10) that will provide accurate simulations without the unnecessary expense of overly increasing the number of time steps in the simulation. However, the falling drop simulations show that the **Minimum Resolved Time Step** flag is needed in order to avoid the death spiral that can occur when the **Time step error** tolerance fails to be met, even for time steps much shorter than the physical and numerical time scales of the problem. By defining these time scales for the problem it becomes much easier to specify the time step parameters to provide cost effective, accurate, level set simulations.

References

Chessa, J., and Belytschko, T., An enriched finite element method and level sets for axisymmetric two-phase flow with surface tension, *International Journal for Numerical Methods in Engineering*, vol. 58, p. 2041-2064, 2003.

Smolianski, A., Finite-element/level-set/operator-splitting (FELSOS) approach for computing two-fluid unsteady flows with free moving interfaces, *International Journal for Numerical Methods in Fluids*, vol. 48, p. 231-269, 2005.